

שיעור 3: תכנון מערכת — משימת כיתה

בניית אפליקציה Full-Stack מונחית AI
יריב גלעד ©

תזכורת מהירה (5 דקות)

לפני שמתחילים לתכנן את המערכת, וודאו שאתם זוכרים את היסודות משיעור 2:

שם האפליקציה	שם זמני - ShiftSync
הבעיה (משפט אחד)	אין מערכת שיודעת לקבוע רצף תורים בצורה נוחה
הפתרון (משפט אחד)	מערכת שיודעת לבדוק זמינות של ספקי שירות לצורך קביעת רצף נוח
פיצירי Must משיעור 2	מנוע הזנת זמינות לעובדים מסך אישור משמרות למנהל טופס הזמנת תור ללקוחות חיצוניים שליחת התראות בסיסיות שיבוץ עובד לתור

מושגי יסוד (10 דקות)

לפני שמשרטטים, חשוב להבין את המושגים הבסיסיים. אלו הלבנים של כל אפליקציה:

מושג	הסבר במשפט
Frontend	מה שרץ בדפדפן של המשתמש. המסך הויזואלי — מסכים, כפתורים, אינטראקציות.
Backend	מה שרץ על השרת. לוגיקה עסקית, אימות, אחסון נתונים, הרשאות.
Database	מקום האחסון הקבוע של הנתונים. טבלאות, שורות, עמודות, קשרים ביניהם.
API	הדרך שבה Frontend מדבר עם Backend. שולח בקשה, מקבל תשובה.
שירותי צד שלישי	כלים מוכנים שלא אתם בונים: אימות (Auth), תשלומים (Stripe), מיילים (Resend), מפות (Maps).

הטק סטאק שלנו (5 דקות)

טכנולוגיה	תפקיד	שכבה
Vite + React	Frontend	UI & View
Supabase (PostgreSQL, Auth, Storage, Edge Functions)	Backend + DB	Logic & Data
(Vercel (Frontend) + Supabase Cloud (Backend	Deploy	Hosting
GitHub	Source Control	Ver. Management

מפת מסכים (15 דקות)



רשמו את כל המסכים (דפים) שהאפליקציה שלכם תכלול. חשבו על המסע מנקודת המבט של המשתמש:

טיפ: התחילו מהמסכים הבסיסיים — דף בית (Home), התחברות/התנתקות (Login/Register), המסך הראשי של האפליקציה (Dashboard), ופרופיל.

#	שם המסך / דף	מה המשתמש עושה פה?	אילו נתונים צריך?
1	מסך התחברות (Login)	מנהלים ועובדים מתחברים למערכת	אימייל וסיסמה (Auth) או התחברות Social
2	דאשבורד מנהל - תצוגה מנהלית (Manager Home)	רואים במבט על את כל התורים שנקבעו השבוע ואת מצב האיוש שלהם (מאויש / חסר עובד).	רשימת תורים (Appointments), רשימת משמרות מאושרות (Shifts). רשימת דרישה לעובדים בימים מסויימים בהתאם לתורים עתידיים.
3	מסך שיבוץ ואישור משמרת (Shift Assignment)	המשתמש מגדיר משמרות לעובדים, ורואה דרישות.	פרטי משמרת או יום ספציפי, רשימת עובדים + סטטוס הזמינות שלהם פר שעה.
4	ממשק זמינות עובד (Employee Availability UI)	ממשק מותאם למובייל בו עובד מסמן אילו ימים ושעות הוא פנוי לעבוד בשבוע הבא.	תאריכים של השבוע הבא ונוכחי, משבצות זמן (Time slots), מזהה עובד.
5	אזור אישי עובד - המשמרות שלי (My Shifts)	למרות הסנכרון ליומן, עובד יכול להיכנס ולראות רשימה מסודרת של כל המשמרות שלו לחודש הקרוב.	טבלת משמרות (Shifts) מסוננת לפי ה-ID, עם סטטוסים.
6	טופס הזמנת תור ללקוח (Customer Booking)	לקוחה נכנסת דרך קישור, רואה זמנים פנויים וקובעת תורים ללא צורך בהורדת אפליקציה.	חיתוך של שעות פעילות העסק פחות שעות שכבר נתפסו (Available Slots).
7	מסך אישור הזמנה (Booking Success)	לקוחה מקבלת חייווי שהתור נקבע בהצלחה והודעה שתישלח אליה תזכורת במייל או בוואצאפ בקרוב.	מזהה הזמנה (Booking ID), תאריך ושעה של התור, פרטי הלקוח שהוזנו.

זרימת נתונים (15 דקות) →

בחרו 3–4 פעולות מרכזיות שהמשתמש עושה באפליקציה, ותארו מה קורה בכל שכבה:

דוגמה: משתמש לוחץ על „הוסף מתכון“ → Frontend שולח בקשה ל'Backend' → Supabase שומר בטבלה → המסך מתעדכן ומראה את הפריט החדש.

#	פעולת משתמש	מה קורה ב-Frontend?	מה קורה ב-Backend?	אילו נתונים נשמרים?
1	התחברות למערכת (Login / Social)	המשתמש בוחר "התחבר עם Google" או מזין אימייל וסיסמה. ה-Frontend קורה לבקשת הזדהות בבק-אנד.	מאמת את הנתונים מול ספק הזהויות (Google/Email), מנפיק Session Token ומחזיר ל-Client.	מזהה משתמש (User ID), תפקיד (Role), ו-Token התחברות פעיל.
2	הלקוחה קובעת תור (Customer Booking)	הלקוחה בוחרת שעה פנויה, מזינה פרטים ולוחצת "הזמן". המסך עובר ל-Booking Success.	ה-API מוודא שהתור לא נתפס, שומר ב-Database, ומפעיל Webhook לאוטומציה (לשליחת תזכורת עתידית בוואטסאפ/מייל).	פרטי התור (תאריך, שעה), מזהה הלקוחה (שם, טלפון, אימייל), סטטוס תור (פעיל).
3	העובד מזין זמינות (Employee Availability)	העובד מסמן במובייל משבצות זמן (Time slots) בהן הוא פנוי ולוחץ "שמור".	מקבל את הבקשה, בודק הרשאות (Token), ומעדכן/מוסיף שורות בטבלת הזמינות של העובד מול היום.	מזהה עובד, תאריכים, שעות התחלה וסיום של הזמינות שהוזנה.
	המנהל משבץ משמרת (Shift Assignment)	המנהל רואה את דרישת העובדים מול התורים, בוחר עובד פנוי ולוחץ "אשר משמרת". ה-UI מתעדכן למצב "מאושר".	שומר את המשמרת ב-DB. מפעיל Edge Function / Webhook ששולח התראה לעובד (בוואטסאפ) ומסנכרן את המשמרת ליומן שלו.	מזהה משמרת, מזהה עובד (Employee ID), מזהה תור מקושר, וסטטוס משמרת ("אושר").

שירותים חיצוניים (10 דקות)

לא כל דבר צריך לבנות מאפס. סמנו אילו שירותים מוכנים האפליקציה שלכם תשתמש בהם:

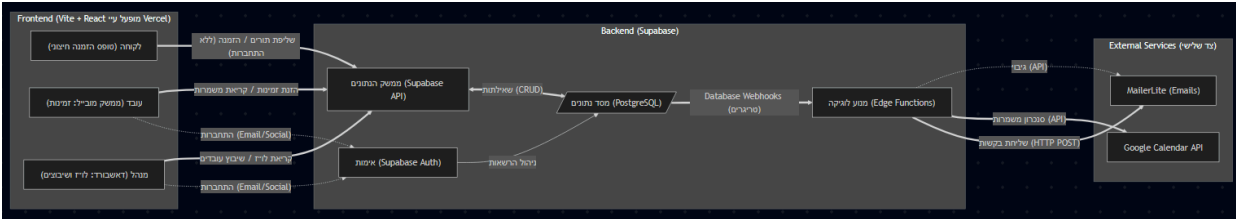
שירות	צריך את זה?	אילו ספק / כלי?
אימות (Authentication)	כן	Supabase / Google
אחסון קבצים (Storage)	לא	
תשלומים (Payments)	לא	
מיילים (Emails)	כן	MailerLite

דיאגרמת ארכיטקטורה (20 דקות)

עכשיו שרטטו את המערכת שלכם. ציירו קופסאות לכל רכיב, חיצים לכל חיבור, וסמנו את כיוון הזרימה.

טיפ: אם אתם מעדיפים לצייר על נייר, השתמשו ב*AI* כדי לייצר דיאגרמה ואז העתיקו אותה לכאן. אפשר גם להשתמש ב*Napkin* או *Excalidraw*.

שרטטו כאן את דיאגרמת הארכיטקטורה שלכם



תכנון מערכת עם 15 AI דקות

השתמשו בעוזר AI כדי לוודא ולהשלים את התכנון שלכם. נסו פרומפטים כמו:

- „אני בונה אפליקציה ל[X]. הטקסט שלי הוא Vite + React, Supabase, Vercel. צור לי דיאגרמת ארכיטקטורה של המערכת עם כל הרכיבים העיקריים.“
- „אילו מסכים (דפים) האפליקציה שלי צריכה? מה המשתמש רואה בכל אחד? אילו נתונים צריך כל מסך?“
- „תאר את זרימת הנתונים הראשית של האפליקציה שלי: מה הפעולות העיקריות שהמשתמש עושה, ומה קורה בכל שכבה (frontend, backend, database)?“

מה גיליתם? האם הציג רכיבים שלא חשבתם עליהם?

טעויות נפוצות של מתחילים

טעות נפוצת	מה לעשות במקום
להוסיף שירותים שלא צריך v1	להתחיל עם המינימום ההכרחי ולהוסיף אחר כך
לדלג על דיאגרמת ארכיטקטורה	לשרטט את המערכת לפני שכותבים שורה אחת של קוד
לבחור טכנולוגיות אקזוטיות „כי זה נשמע מגניב“	להישאר עם הסטאק שה-AI מכיר ויש לו קהילה חזקה

סיכום וצעדים הבאים

לפני שעוזבים, וודאו שהשלמתם:

מפת מסכים מלאה	☐
טבלת זרימת נתונים ל-3–4 פעולות	☐
רשימת שירותים חיצוניים	☐
דיאגרמת ארכיטקטורה משורטטת	☐
ווידאו מול-AI — התכנון הגיוני?	☐

בשיעור 4 נתחיל לשרטט מסכים (Wireframe). הדיאגרמה שלכם היא המפה — עכשיו נמלא אותה בתוכן.